

PROJECT DOCUMENTATION

Smart Curriculum Activity & Attendance App

Phase 1: Planning & Design | June 01 – 09, 2025

Domain: Education Technology / AI / Mobile & Web Application

Technology Stack: React Native / Flutter | Python (Django/Flask)

Firebase | SQLite / MySQL | AI/ML | REST APIs

Activity Summary — June 01 to June 09, 2025

Day	Date	Activity	Status
Day 1	June 01	Project Title Finalization	✓ Completed
Day 2	June 02	Requirement Gathering	✓ Completed
Day 3	June 03	Objective Definition	✓ Completed
Day 4	June 04	User & Module Identification	✓ Completed
Day 5	June 05	Use Case Diagram Preparation	✓ Completed
Day 6	June 06	Database Requirement Analysis	✓ Completed
Day 7	June 07	ER Diagram Design	✓ Completed
Day 8	June 08	Database Schema Creation	✓ Completed
Day 9	June 09	UI Wireframe Design	✓ Completed

Project Title

Smart Curriculum Activity & Attendance App

Domain

Education Technology / Artificial Intelligence / Mobile & Web-Based Application

Technology Stack

- ➤ Frontend / Mobile: React Native / Flutter
- ➤ Backend: Python (Django / Flask)
- ➤ AI/ML: Scikit-learn / TensorFlow / NLP libraries
- ➤ Database: SQLite / MySQL / Firebase
- ➤ APIs: REST APIs / Google Calendar API / Notification Services

Project Overview

The Smart Curriculum Activity & Attendance App is designed to streamline academic management for educational institutions by automating attendance tracking, managing curriculum activities, and providing intelligent insights to educators, students, and administrators. The system leverages AI/ML for predictive analytics, pattern detection, and personalized recommendations — reducing manual effort and improving institutional efficiency.

Outcome: Title approved: Smart Curriculum Activity & Attendance App. Scope confirmed with guide.

Functional Requirements

The system should be able to:

- ➤ Allow teachers to mark and manage daily attendance digitally
- ➤ Enable students to view their own attendance records and activity schedules
- ➤ Support curriculum planning and activity scheduling by staff
- ➤ Send automated notifications/alerts for low attendance or upcoming activities
- ➤ Generate attendance reports (daily, weekly, monthly) for administrators
- ➤ Provide AI-driven insights on student engagement and attendance patterns
- ➤ Allow admin to manage users, departments, courses, and timetables
- ➤ Support multi-role login: Admin, Teacher, Student, Parent
- ➤ Integrate with academic calendar and holiday management

Non-Functional Requirements

- ➤ Performance: Dashboard loads within 2–3 seconds; attendance submission in real time
- ➤ Accuracy: Attendance and schedule data must be consistent and tamper-proof
- ➤ Usability: Intuitive mobile-first UI requiring minimal training
- ➤ Scalability: Supports 10,000+ students and 500+ staff simultaneously
- ➤ Security: Role-based access control; data encrypted at rest and in transit
- ➤ Availability: 99.5% uptime; offline mode for attendance marking
- ➤ Maintainability: Modular codebase allowing easy updates to curriculum or policies

Hardware Requirements

- ➤ Processor: i3 or higher (recommended i5/i7 for server deployment)
- ➤ RAM: Minimum 4 GB (8 GB preferred for AI processing modules)
- ➤ Storage: 1 GB – 5 GB depending on dataset and media files
- ➤ Internet connection (for cloud sync, notifications, and API access)
- ➤ Mobile devices: Android 8+ / iOS 13+ for the mobile app

Software Requirements

- ➤ Python 3.9+
- ➤ Mobile framework: React Native / Flutter
- ➤ Backend framework: Django / Flask
- ➤ ML libraries: Scikit-learn / TensorFlow / Pandas / NumPy
- ➤ Database: SQLite (local) / MySQL (production) / Firebase (real-time sync)
- ➤ Push notifications: Firebase Cloud Messaging (FCM)
- ➤ Frontend: React.js (admin web portal)

User Requirements

- ➤ Teachers should be able to mark attendance in a single tap
- ➤ Students should view schedules and attendance without technical knowledge
- ➤ Parents should receive timely notifications about their child's attendance
- ➤ Admin should access reports and analytics from a single dashboard

Outcome: Complete requirements document prepared covering all modules, user roles, and system constraints.

Main Objective

To develop an intelligent, mobile-first application that automates attendance management, streamlines curriculum activity planning, and provides AI-driven insights to educators and administrators — enhancing academic efficiency and student accountability.

Primary Objectives

- ➤ Automate the daily attendance process for teachers and students
- ➤ Provide a centralized platform for curriculum activity scheduling and tracking
- ➤ Generate real-time attendance reports and analytics for administrators
- ➤ Alert stakeholders (teachers, parents, students) for low attendance or missed activities
- ➤ Enable AI-based prediction of at-risk students based on attendance trends
- ➤ Support offline attendance marking with automatic sync on reconnection

Secondary Objectives

- ➤ To reduce paperwork and manual record-keeping in educational institutions
- ➤ To improve transparency between school administration, teachers, and parents
- ➤ To provide data-driven insights for curriculum improvement
- ➤ To enable institutions to comply with attendance policies through automated tracking

Outcome: Objectives defined and documented. Scope confirmed to focus on attendance automation, curriculum activity management, real-time reporting, and AI-driven student insights.

User Identification

1. Students

- ➤ Primary end users of the mobile app
- ➤ View personal attendance records and academic activity schedules
- ➤ Receive notifications for upcoming activities or attendance shortfalls
- ➤ No technical knowledge required

2. Teachers / Faculty

- ➤ Mark attendance for their assigned classes using the mobile app
- ➤ Schedule and manage curriculum activities and assignments
- ➤ View class-wise attendance summaries and generate reports
- ➤ Access AI insights on student engagement patterns

3. Parents / Guardians

- ➤ Monitor their child's attendance and activity participation remotely
- ➤ Receive automated alerts for low attendance or missed sessions
- ➤ Communicate with teachers through the platform

4. System Administrator

- ➤ Manage users, departments, courses, and academic schedules
- ➤ Configure attendance policies and notification thresholds
- ➤ Access institution-wide analytics and reports
- ➤ Maintain system security, data integrity, and performance

Module Identification

1. Authentication & User Management Module

- ➤ Handles multi-role login (Admin, Teacher, Student, Parent)
- ➤ Manages user profiles, permissions, and session tokens
- ➤ Supports password reset and secure onboarding

2. Attendance Management Module

- ➤ Allows teachers to mark, edit, and submit daily attendance
- ➤ Supports class-wise, subject-wise, and date-range filtering
- ➤ Provides offline marking with background sync

3. Curriculum & Activity Scheduling Module

- ➤ Enables teachers and admins to create and publish activity schedules
- ➤ Manages timetables, academic events, and deadlines
- ➤ Integrates with the calendar for conflict detection

4. AI Analytics & Insights Module

- ➤ Detects at-risk students using attendance trend analysis
- ➤ Generates predictive reports for academic performance
- ➤ Surfaces anomalies and patterns for admin review

5. Notification & Alert Module

- ➤ Sends push notifications for low attendance, upcoming activities

- ➤ Delivers SMS/email alerts to parents for critical updates
- ➤ Supports configurable alert thresholds per institution policy

6. Reporting & Dashboard Module

- ➤ Generates daily, weekly, and monthly attendance reports
- ➤ Provides role-specific dashboards with charts and summaries
- ➤ Supports export to PDF or Excel

7. Database Management Module

- ➤ Stores all user, attendance, and activity data securely
- ➤ Manages real-time sync via Firebase and batch processing via MySQL
- ➤ Supports data backup and archival policies

8. Admin Control Module

- ➤ Allows admin to configure institution settings and policies
- ➤ Monitors system health and user activity logs
- ➤ Manages course structures, academic years, and departments

Outcome: 4 user roles and 8 system modules identified and documented.

Actors

- ➤ Student
- ➤ Teacher / Faculty
- ➤ Parent / Guardian
- ➤ Admin (System Administrator)

Use Cases

Student Use Cases

- ➤ Login / Register
- ➤ View Personal Attendance Record
- ➤ View Curriculum Activity Schedule
- ➤ Receive Attendance Alerts
- ➤ Download Attendance Report
- ➤ Provide Feedback

Teacher Use Cases

- ➤ Login to Teacher Panel
- ➤ Mark / Update Class Attendance
- ➤ Schedule Curriculum Activities
- ➤ View Class Attendance Summary
- ➤ Generate & Export Attendance Report
- ➤ Receive AI Insights on Student Patterns

Parent Use Cases

- ➤ Login / Register
- ➤ Monitor Child Attendance
- ➤ Receive Low-Attendance Alerts
- ➤ View Child Activity Schedule

Admin Use Cases

- ➤ Login to Admin Panel
- ➤ Manage Users and Departments
- ➤ Configure Attendance Policies
- ➤ Manage Academic Calendar
- ➤ View System Analytics & Reports
- ➤ Monitor System Performance

UML Diagrams Designed

1. Use Case Diagram

Represents interactions between 4 actors and the system across all functional areas.

2. Class Diagram

Main classes: User, Student, Teacher, Admin, Parent, AttendanceRecord, CurriculumActivity, Notification, Report — each with defined attributes and methods.

3. Activity Diagram

Flow: Teacher logs in → Selects class → Marks attendance → System validates → Saves to DB → Triggers notification if threshold breached → Report updated.

4. Sequence Diagram

Participants: User, Mobile App, Auth Service, Attendance Service, Notification Service, Database. Shows end-to-end message flow for attendance submission.

5. Deployment Diagram

Components: Student/Teacher Device → Mobile App (React Native/Flutter) → Backend Server (Django + AI Module) → Database Server (MySQL/Firebase).

Outcome: 20 use cases identified across 4 actors. All UML diagrams (use case, class, activity, sequence, deployment) prepared and reviewed.

Data Requirements & Classifications

The architecture supports four primary types of data storage:

1. Attendance & Academic Data (Relational Layer)

- ➤ Student and teacher profiles, class/course mappings
- ➤ Daily attendance records with timestamps and session info
- ➤ Curriculum activity logs, deadlines, and completion status

2. Real-Time Sync Data (NoSQL / Firebase Layer)

- ➤ Live attendance state, active session tokens
- ➤ Push notification queues, online/offline device states
- ➤ Parent monitoring feeds and real-time dashboard updates

3. AI & Analytics Data (Time-Series / Analytical Layer)

- ➤ Historical attendance trends for predictive modelling
- ➤ Engagement scores and activity participation metrics
- ➤ Anomaly detection logs and model output metadata

4. Configuration & Operational Data (Relational Layer)

- ➤ Institution settings, department structures, academic calendars
- ➤ Attendance policy thresholds, notification configurations
- ➤ System audit logs and admin action history

Entities and Attributes

User

- User_ID (PK), Full_Name, Email, Phone, Role, Password_Hash, Created_At

Student

- Student_ID (PK), User_ID (FK), Roll_Number, Department, Year, Section

Teacher

- Teacher_ID (PK), User_ID (FK), Employee_ID, Department, Subjects-Taught

Course

- Course_ID (PK), Course_Name, Course_Code, Department, Semester, Credits

Attendance_Record

- Record_ID (PK), Student_ID (FK), Course_ID (FK), Teacher_ID (FK), Date, Status (Present/Absent/Late), Marked_At

Curriculum_Activity

- Activity_ID (PK), Title, Description, Course_ID (FK), Teacher_ID (FK), Scheduled_Date, Due_Date, Activity_Type, Status

Notification

- Notif_ID (PK), User_ID (FK), Message, Type, Sent_At, Is_Read

Report

- Report_ID (PK), Generated_By (FK), Report_Type, Date_Range, File_Path, Created_At

Feedback

- Feedback_ID (PK), User_ID (FK), Rating, Comments, Submitted_At

Non-Functional Database Requirements

- ➤ Low Latency: Attendance queries return within milliseconds; reports < 3 seconds
- ➤ Security: AES-256 encryption at rest; TLS 1.3 in transit; PII handled per PDPA/FERPA
- ➤ Offline Availability: SQLite local schema for offline attendance; sync on reconnect
- ➤ Scalability: Horizontal sharding support for 100,000+ student records during peak periods

**Outcome: 9 entities identified with all key attributes and inter-entity relationships mapped.
Polyglot persistence model (Relational + NoSQL + Time-Series) confirmed.**

Entities and Attributes

User

- ➤ User_ID (PK) — Primary Key
- ➤ Full_Name, Email, Phone, Role, Password_Hash, Created_At

Student

- ➤ Student_ID (PK) — Primary Key
- ➤ Roll_Number, Department, Year, Section
- ➤ User_ID (FK → User)

Teacher

- ➤ Teacher_ID (PK) — Primary Key
- ➤ Employee_ID, Department, Subjects-Taught
- ➤ User_ID (FK → User)

Course

- ➤ Course_ID (PK) — Primary Key
- ➤ Course_Name, Course_Code, Department, Semester, Credits

Attendance_Record

- ➤ Record_ID (PK) — Primary Key
- ➤ Date, Status, Marked_At
- ➤ Student_ID (FK → Student), Course_ID (FK → Course), Teacher_ID (FK → Teacher)

Curriculum_Activity

- ➤ Activity_ID (PK) — Primary Key
- ➤ Title, Description, Scheduled_Date, Due_Date, Activity_Type, Status
- ➤ Course_ID (FK → Course), Teacher_ID (FK → Teacher)

Notification

- ➤ Notif_ID (PK) — Primary Key
- ➤ Message, Type, Sent_At, Is_Read
- ➤ User_ID (FK → User)

Feedback

- ➤ Feedback_ID (PK) — Primary Key
- ➤ Rating, Comments, Submitted_At
- ➤ User_ID (FK → User)

Report

- ➤ Report_ID (PK) — Primary Key
- ➤ Report_Type, Date_Range, File_Path, Created_At
- ➤ Generated_By (FK → User)

ER Flow

Student → has → Attendance_Record → linked to → Course → assigned to → Teacher

Teacher → creates → Curriculum_Activity → belongs to → Course

User → receives → Notification

User → submits → Feedback

User → generates → Report

Outcome: ER diagram designed with 9 entities, all primary/foreign keys defined, and 8 relationships mapped.

Overview

The database schema for the Smart Curriculum Activity & Attendance App consists of nine main tables storing user profiles, attendance records, course details, activity schedules, notifications, feedback, and generated reports.

SQL Schema

Create Database

```
CREATE DATABASE smart_curriculum_app;  
USE smart_curriculum_app;
```

1. Users Table

```
CREATE TABLE users (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  phone VARCHAR(20),  
  password VARCHAR(255) NOT NULL,  
  role ENUM('admin', 'teacher', 'student', 'parent')  
    DEFAULT 'student',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. Students Table

```
CREATE TABLE students (  
  student_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  roll_number VARCHAR(20) UNIQUE NOT NULL,  
  department VARCHAR(100),  
  year INT,  
  section VARCHAR(10),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

3. Teachers Table

```
CREATE TABLE teachers (  
  teacher_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  employee_id VARCHAR(20) UNIQUE NOT NULL,  
  department VARCHAR(100),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

4. Courses Table

```
CREATE TABLE courses (  
  course_id INT AUTO_INCREMENT PRIMARY KEY,  
  course_name VARCHAR(150) NOT NULL,  
  course_code VARCHAR(20) UNIQUE NOT NULL,  
  department VARCHAR(100),  
  semester INT,  
  credits INT,  
  teacher_id INT,  
  FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)  
);
```

5. Attendance Records Table

```
CREATE TABLE attendance_records (  
  record_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_id INT,  
  course_id INT,  
  teacher_id INT,  
  date DATE NOT NULL,  
  status ENUM('present', 'absent', 'late')  
    DEFAULT 'absent',  
  marked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (student_id) REFERENCES students(student_id),  
  FOREIGN KEY (course_id) REFERENCES courses(course_id),  
  FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)  
);
```

6. Curriculum Activities Table

```
CREATE TABLE curriculum_activities (  
  activity_id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  course_id INT,  
  teacher_id INT,  
  scheduled_date DATE,  
  due_date DATE,  
  activity_type ENUM('lecture', 'assignment', 'exam',  
    'event', 'workshop') DEFAULT 'lecture',  
  status ENUM('upcoming', 'ongoing', 'completed')  
    DEFAULT 'upcoming',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (course_id) REFERENCES courses(course_id),  
  FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)
```



```
);
```

7. Notifications Table

```
CREATE TABLE notifications (  
  notif_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  message TEXT NOT NULL,  
  type ENUM('attendance', 'activity', 'system', 'alert')  
      DEFAULT 'system',  
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  is_read BOOLEAN DEFAULT FALSE,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

8. Feedback Table

```
CREATE TABLE feedback (  
  feedback_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  rating INT CHECK (rating BETWEEN 1 AND 5),  
  comments TEXT,  
  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

9. Reports Table

```
CREATE TABLE reports (  
  report_id INT AUTO_INCREMENT PRIMARY KEY,  
  generated_by INT,  
  report_type ENUM('attendance', 'activity', 'analytics')  
      DEFAULT 'attendance',  
  date_from DATE,  
  date_to DATE,  
  file_path VARCHAR(300),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (generated_by) REFERENCES users(user_id)  
);
```

Outcome: Database schema creation completed. All 9 tables created and verified in MySQL. Foreign key relationships established across users, students, teachers, courses, attendance_records, curriculum_activities, notifications, feedback, and reports.

Design Overview

Design Style: Clean, modern education dashboard with blue and white color theme, mobile-first responsive layout, card-based components, and minimal navigation depth.

Screen 1: Home / Splash Screen

- ➤ App Logo and Name: Smart Curriculum Activity & Attendance App
- ➤ Tagline: 'Smart Learning. Smarter Tracking.'
- ➤ Login Button (primary CTA)
- ➤ Register / Sign Up link
- ➤ Institution Code Entry (for multi-school deployment)

Screen 2: Multi-Role Login / Registration

- ➤ Role selector: Student | Teacher | Parent | Admin
- ➤ Login Form: Email / Roll Number / Employee ID + Password
- ➤ Registration Form: Name, Email, Phone, Role, Institution Code
- ➤ Forgot Password link

Screen 3: Student Dashboard

- ➤ Header: Student Name + Notification Bell + Profile Icon
- ➤ Attendance Summary Card: Present % | Absent | Late
- ➤ Today's Schedule: Upcoming classes and activity cards
- ➤ Quick Links: View Full Attendance | Download Report | Feedback
- ➤ Bottom Navigation: Home | Attendance | Schedule | Profile

Screen 4: Teacher — Attendance Marking Interface

- ➤ Class Selector: Department → Year → Section → Subject
- ➤ Date Picker (defaults to today)
- ➤ Student List with one-tap Present / Absent / Late toggle
- ➤ Bulk Mark All Present button
- ➤ Submit Attendance button with confirmation dialog
- ➤ Offline mode indicator and sync status badge

Screen 5: Curriculum Activity Scheduler

- ➤ Calendar View: Monthly / Weekly toggle with activity dots
- ➤ Add Activity FAB button: Title, Type, Course, Date, Description
- ➤ Activity Cards: Colour-coded by type (Lecture / Assignment / Exam / Event)
- ➤ Filter by Course / Date Range / Activity Type
- ➤ Edit and Delete options per activity card

Screen 6: Admin Dashboard (Web Portal)

- ➤ Sidebar Navigation: Dashboard | Users | Attendance | Courses | Activities | Reports | Settings
- ➤ Dashboard Metrics: Total Students | Active Teachers | Today's Attendance % | Pending Activities
- ➤ Charts & Analytics: Attendance Trend Graph | Course-wise Attendance Bar Chart | Monthly Activity Report
- ➤ Quick Actions: Add User | Import Data | Generate Report | Configure Alerts

Outcome: UI wireframes designed for all 6 screens covering the complete user journey from splash screen to admin web portal. Mobile-first layout with blue and white education theme finalized and ready for development.